

ClassConnect — Full Python (app.py) Code Guide

Line-by-Line & Function-by-Function Explanation for CS50 Final Project

Overview: This guide breaks down every single module import, configuration setting, custom decorator, route function, SQL query, and design pattern inside app.py. Each section explains *what it does*, *why it is required*, and *how it connects with your templates and database*.

1. Modules & Import Statements (Lines 1–10)

Every import brings pre-built tools into Python so we don't have to reinvent the wheel:

- **import os:** Interacts with the operating system (paths, creating folders with `os.makedirs`).
- **import random:** Generates pseudo-random choices (used to build 6-character class join codes).
- **import string:** Supplies character sets like uppercase letters `string.ascii_uppercase` and digits.
- **from functools import wraps:** Preserves original function names and docstrings when creating custom decorators.
- **from flask import Flask, flash, redirect, render_template, request, session, url_for:** Core Flask tools: web server (Flask), user alerts (flash), URL jumps (redirect), HTML rendering (render_template), form data reader (request), user memory (session), URL builder (url_for).
- **from flask_session import Session:** Stores session data server-side (in files) rather than inside insecure browser cookies.
- **from cs50 import SQL:** Executes SQL queries with automatic SQL-injection prevention using '?' placeholders.
- **from werkzeug.security import check_password_hash, generate_password_hash:** Security tools: hashes passwords with cryptographic salt so plaintext passwords are never stored.
- **from werkzeug.utils import secure_filename:** Strips dangerous path characters ('../..') from uploaded file names to avoid directory traversal attacks.
- **from datetime import datetime:** Gets current date and time (used for timestamps on uploaded files and announcements).

2. Application Setup & Configuration (Lines 12–30)

```
app = Flask(__name__)
app.config["TEMPLATES_AUTO_RELOAD"] = True
app.config["SESSION_PERMANENT"] = False
app.config["SESSION_TYPE"] = "filesystem"
Session(app)
db = SQL("sqlite:///classconnect.db")
UPLOAD_FOLDER = os.path.join('static', 'uploads')
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
```

Breakdown of settings:

- **Flask(__name__)**: Instantiates the Flask WSGI application. `__name__` tells Flask where to locate templates and static files relative to the current script.
- **TEMPLATES_AUTO_RELOAD = True**: Re-renders HTML templates automatically whenever you edit them without requiring a manual server restart.
- **SESSION_PERMANENT = False & SESSION_TYPE = 'filesystem'**: Sessions expire when the browser window closes. Session variables (like `user_id`) are saved securely on the server's disk rather than in client-side cookies.
- **Session(app)**: Activates server-side session management for this app instance.
- **db = SQL('sqlite:///classconnect.db')**: Connects the CS50 SQL engine to the SQLite database file.
- **UPLOAD_FOLDER & os.makedirs(..., exist_ok=True)**: Configures static/uploads as the upload target and guarantees the folder exists on disk.

3. Caching Control & Custom Decorators (Lines 31–60)

A. Cache Control: @app.after_request

```
@app.after_request
def after_request(response):
    response.headers["Cache-Control"] = "no-cache, no-store, must-revalidate"
    response.headers["Expires"] = 0
    response.headers["Pragma"] = "no-cache"
    return response
```

Purpose: Forces web browsers never to cache pages locally. If a user logs out and hits the browser's 'Back' button, they won't see confidential student or teacher data from cache.

B. Authentication Guard: @login_required

```
def login_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if session.get("user_id") is None:
            return redirect("/login")
        return f(*args, **kwargs)
    return decorated_function
```

Purpose: Protects private routes. Before executing the view function `f`, it checks if `session['user_id']` is set. If not logged in, redirects immediately to `/login`. The `@wraps(f)` decorator ensures Flask keeps the original function name for route resolution.

C. Role Authorization Guard: @teacher_required

```
def teacher_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if session.get("role") != 'teacher':
            flash("You must be a teacher to access this page.", "danger")
            return redirect("/")
        return f(*args, **kwargs)
    return decorated_function
```

Purpose: Role-based access control. If a logged-in student tries to manually browse to a teacher URL (e.g. /create_class or /attendance), this blocks them, flashes an error alert, and returns them to their homepage.

4. Route Functions Breakdown (Line-by-Line)

Route 1: GET / — index() (Lines 62–73)

```
@app.route("/")
@login_required
def index():
    if session["role"] == "teacher":
        classes = db.execute("SELECT * FROM classes WHERE teacher_id = ?", session["user_id"])
        return render_template("teacher_dashboard.html", classes=classes)
    else:
        classes = db.execute("SELECT classes.* FROM classes JOIN enrollments ON classes.id = enrollments.class_id")
        return render_template("student_dashboard.html", classes=classes)
```

- **What it does:** The main homepage. First verifies login.
- **Teacher Branch:** Queries all classes created by this teacher (WHERE teacher_id = ?) and renders teacher_dashboard.html.
- **Student Branch:** Uses an SQL JOIN between classes and enrollments to find all classes the student has joined, rendering student_dashboard.html.

Route 2 & 3: /login & /logout (Lines 74–110)

```
@app.route("/login", methods=["GET", "POST"])
def login():
    session.clear()
    if request.method == "POST":
        username = request.form.get("username")
        password = request.form.get("password")
        # Validate inputs ...
        rows = db.execute("SELECT * FROM users WHERE username = ?", username)
        if len(rows) != 1 or not check_password_hash(rows[0]["hash"], password):
            flash("Invalid username and/or password", "danger")
            return render_template("login.html")
        session["user_id"] = rows[0]["id"]
        session["username"] = rows[0]["username"]
        session["role"] = rows[0]["role"]
        return redirect("/")
    return render_template("login.html")
```

- **session.clear():** Clears previous login tokens to prevent credential hijacking.
- **request.form.get(...):** Safely reads data submitted through the HTML <form> POST request.
- **check_password_hash(rows[0]['hash'], password):** Cryptographically verifies the entered plain password against the stored bcrypt/scrypt hash in the database.
- **session['user_id'], ['role']:** Remembers who is logged in across subsequent HTTP requests.
- **logout():** Simply calls session.clear() and redirects back to /.

Route 4: GET / POST /register — register() (Lines 111–144)

```
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        # Check blank fields, password matching, valid role ('teacher'/'student')
        hash_pw = generate_password_hash(password)
        try:
            user_id = db.execute("INSERT INTO users (username, hash, role) VALUES (?, ?, ?)", username, hash_pw, role)
        except ValueError:
            flash("Username already taken.", "danger")
            return render_template("register.html")
        session["user_id"] = user_id
        session["username"] = username
        session["role"] = role
        return redirect("/")
    return render_template("register.html")
```

- **Input validation:** Checks for empty inputs, verifies password confirmation equality, and ensures role is either 'teacher' or 'student'.
- **generate_password_hash(password):** Hashes the password before touching the database.
- **try / except ValueError:** In CS50 SQL, inserting a duplicate entry on a UNIQUE column (like username) raises a ValueError. We catch this to inform the user nicely instead of crashing with a 500 error.
- **Auto-login:** Sets the session immediately upon registration and redirects to the dashboard.

Route 5 & 6: /create_class & /join_class (Lines 145–190)

```
# Create Class (Teachers only)
join_code = ''.join(random.choices(string.ascii_uppercase + string.digits, k=6))
db.execute("INSERT INTO classes (name, teacher_id, join_code) VALUES (?, ?, ?)",
           class_name, session["user_id"], join_code)

# Join Class (Students only)
class_info = db.execute("SELECT * FROM classes WHERE join_code = ?", join_code)
# Check already enrolled ...
db.execute("INSERT INTO enrollments (student_id, class_id) VALUES (?, ?)",
           session["user_id"], class_info[0][1])
```

- **Join Code Generation:** random.choices(..., k=6) picks 6 random uppercase letters/numbers and joins them into a clean string like 'AB39K2'.
- **Join Class Safety:** Verifies the code exists, prevents duplicate enrollments with a query check, and inserts the link into the enrollments join table.

Route 7 & 8: /class/<id> & /announce (Lines 191–227)

```
@app.route("/class/<int:class_id>")
@login_required
def class_view(class_id):
    if session["role"] == "teacher":
        class_info = db.execute("SELECT * FROM classes WHERE id = ? AND teacher_id = ?", class_id, session["user_id"])
    else:
        class_info = db.execute("SELECT classes.* FROM classes JOIN enrollments ON classes.id = enrollments.class_id WHERE enrollments.student_id = ?", session["user_id"])
    if len(class_info) == 0:
        flash("Class not found or unauthorized.", "danger")
        return redirect("/")
    announcements = db.execute("SELECT * FROM announcements WHERE class_id = ? ORDER BY timestamp DESC", class_id)
    materials = db.execute("SELECT * FROM materials WHERE class_id = ? ORDER BY upload_time DESC", class_id)
    return render_template("class_view.html", class_info=class_info[0], announcements=announcements, materials=materials)
```

- **URL Parameter <int:class_id>:** Captures the class ID integer dynamically from the URL.
- **Strict Authorization:** Crucial security feature! A teacher can only open a class if `teacher_id = session['user_id']`. A student can only open if enrolled. Strangers get kicked out with an alert.
- **announcements & materials:** Queried sorted with newest first (ORDER BY ... DESC).
- **/class/<id>/announce:** Verifies class ownership, extracts announcement text from `request.form.get('text')`, and saves it into the database.

Route 9: POST /class/<id>/upload — Materials (Lines 228–259)

```
@app.route("/class/<int:class_id>/upload", methods=["POST"])
@login_required
@teacher_required
def upload(class_id):
    if 'material' not in request.files: ...
    file = request.files['material']
    if file:
        filename = secure_filename(file.filename)
        timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
        unique_filename = f"{timestamp}_{filename}"
        file_path = os.path.join(app.config['UPLOAD_FOLDER'], unique_filename)
        file.save(file_path)
        db.execute("INSERT INTO materials (class_id, filename, file_path) VALUES (?, ?, ?)",
                   class_id, filename, unique_filename)
```

- **request.files['material']:** Reads binary file data sent via multipart/form-data.
- **secure_filename():** Sanitizes file names to prevent attacks like `../../etc/passwd`.
- **Timestamp prefix:** Prevents two files with the same name from overwriting each other on the server's hard drive.
- **file.save(file_path):** Writes the uploaded PDF/document to `static/uploads/`.
- **Database storage:** Saves the human-readable filename and the disk `file_path` so students can download it.

Route 10 & 11: Teacher & Student Attendance (Lines 260–310)

```
# Teacher Attendance (POST)
for student in students:
    status = request.form.get(f"status_{student['id']}")
    if status in ['Present', 'Absent', 'Late']:
        db.execute("DELETE FROM attendance WHERE class_id = ? AND student_id = ? AND date = ?",
                    class_id, student["id"], date)
        db.execute("INSERT INTO attendance (class_id, student_id, date, status) VALUES (?, ?, ?, ?)",
                    class_id, student["id"], date, status)

# Student View (GET)
records = db.execute("SELECT date, status FROM attendance WHERE class_id = ? AND student_id = ? ORDER BY date")
total = len(records)
present = sum(1 for r in records if r["status"] == "Present")
late = sum(1 for r in records if r["status"] == "Late")
```

- **Dynamic Form Input reading:** Reads `status_{student_id}` radio buttons for each enrolled student.
- **Upsert Logic:** Deletes any previous attendance record for that date before inserting the new one, avoiding duplicates when an edit is made.
- **Aggregations:** Python generator expressions (`sum(1 for r in records if ...)`) calculate total attendance statistics instantly for the student view.

5. Summary of Roles & Route Access Matrix

Route / URL	Method	Who Can Access	Function Purpose
/	GET	Logged-in users	Routes to Teacher or Student dashboard
/login	GET, POST	Everyone (public)	Authenticates user credentials
/register	GET, POST	Everyone (public)	Creates new account with selected role
/logout	GET	Logged-in users	Clears active session
/create_class	POST	Teacher only	Generates unique join code and class
/join_class	POST	Student only	Enrolls student via join code
/class/<id>	GET	Enrolled Student/Owner Teacher	Displays announcements & materials
/class/<id>/announce	POST	Owner Teacher only	Posts new announcement message
/class/<id>/upload	POST	Owner Teacher only	Uploads PDF/subject material file
/class/<id>/attendance	GET, POST	Owner Teacher only	Records daily student attendance
/class/<id>/my_attendance	GET	Enrolled Student only	Shows personal attendance summary